

SECTION 3

INSTRUCTION SET SUMMARY

This section briefly describes the M68000 family instruction set, using Motorola's assembly language syntax and notation. It includes instruction set details such as notation and format, selected instruction examples, and an integer condition code discussion. The section concludes with a discussion of floating-point details such as computational accuracy, conditional test definitions, an explanation of the operation table, and a discussion of not-a-numbers (NaNs) and postprocessing.

3.1 INSTRUCTION SUMMARY

Instructions form a set of tools that perform the following types of operations:

Data Movement	Program Control
Integer Arithmetic	System Control
Logical Operations	Cache Maintenance
Shift and Rotate Operations	Multiprocessor Communications
Bit Manipulation	Memory Management
Bit Field Manipulation	Floating-Point Arithmetic
Binary-Coded Decimal Arithmetic	

The following paragraphs describe in detail the instruction for each type of operation. Table 3-1 lists the notations used throughout this manual. In the operand syntax statements of the instruction definitions, the operand on the right is the destination operand.

Table 3-1. Notational Conventions

Single- And Double Operand Operations	
+	Arithmetic addition or postincrement indicator.
–	Arithmetic subtraction or predecrement indicator.
×	Arithmetic multiplication.
÷	Arithmetic division or conjunction symbol.
~	Invert; operand is logically complemented.
Λ	Logical AND
V	Logical OR
⊕	Logical exclusive OR
→	Source operand is moved to destination operand.
↔	Two operands are exchanged.
<op>	Any double-operand operation.
<operand>tested	Operand is compared to zero and the condition codes are set appropriately.
sign-extended	All bits of the upper portion are made equal to the high-order bit of the lower portion.
Other Operations	
TRAP	Equivalent to Format +Offset Word → (SSP); SSP – 2 → SSP; PC → (SSP); SSP – 4 → SSP; SR → (SSP); SSP – 2 → SSP; (Vector) → PC
STOP	Enter the stopped state, waiting for interrupts.
<operand> ₁₀	The operand is BCD; operations are performed in decimal.
If <condition> then <operations> else <operations>	Test the condition. If true, the operations after “then” are performed. If the condition is false and the optional “else” clause is present, the operations after “else” are performed. If the condition is false and else is omitted, the instruction performs no operation. Refer to the Bcc instruction description as an example.
Register Specifications	
An	Any Address Register n (example: A3 is address register 3)
Ax, Ay	Source and destination address registers, respectively.
Dc	Data register D7–D0, used during compare.
Dh, Dl	Data register’s high- or low-order 32 bits of product.
Dn	Any Data Register n (example: D5 is data register 5)
Dr, Dq	Data register’s remainder or quotient of divide.
Du	Data register D7–D0, used during update.
Dx, Dy	Source and destination data registers, respectively.
MRn	Any Memory Register n.
Rn	Any Address or Data Register
Rx, Ry	Any source and destination registers, respectively.
Xn	Index Register

Table 3-1. Notational Conventions (Continued)

Data Format And Type	
+ inf	Positive Infinity
<fmt>	Operand Data Format: Byte (B), Word (W), Long (L), Single (S), Double (D), Extended (X), or Packed (P).
B, W, L	Specifies a signed integer data type (twos complement) of byte, word, or long word.
D	Double-precision real data format (64 bits).
k	A twos complement signed integer (–64 to +17) specifying a number's format to be stored in the packed decimal format.
P	Packed BCD real data format (96 bits, 12 bytes).
S	Single-precision real data format (32 bits).
X	Extended-precision real data format (96 bits, 16 bits unused).
– inf	Negative Infinity
Subfields and Qualifiers	
#<xxx> or #<data>	Immediate data following the instruction word(s).
()	Identifies an indirect address in a register.
[]	Identifies an indirect address in memory.
bd	Base Displacement
ccc	Index into the MC68881/MC68882 Constant ROM
d _n	Displacement Value, n Bits Wide (example: d ₁₆ is a 16-bit displacement).
LSB	Least Significant Bit
LSW	Least Significant Word
MSB	Most Significant Bit
MSW	Most Significant Word
od	Outer Displacement
SCALE	A scale factor (1, 2, 4, or 8 for no-word, word, long-word, or quad-word scaling, respectively).
SIZE	The index register's size (W for word, L for long word).
{offset:width}	Bit field selection.
Register Names	
CCR	Condition Code Register (lower byte of status register)
DFC	Destination Function Code Register
FPcr	Any Floating-Point System Control Register (FPCR, FPSR, or FPIAR)
FPm, FPN	Any Floating-Point Data Register specified as the source or destination, respectively.
IC, DC, IC/DC	Instruction, Data, or Both Caches
MMUSR	MMU Status Register
PC	Program Counter
Rc	Any Non Floating-Point Control Register
SFC	Source Function Code Register
SR	Status Register

Table 3-1. Notational Conventions (Concluded)

Register Codes	
*	General Case
C	Carry Bit in CCR
cc	Condition Codes from CCR
FC	Function Code
N	Negative Bit in CCR
U	Undefined, Reserved for Motorola Use.
V	Overflow Bit in CCR
X	Extend Bit in CCR
Z	Zero Bit in CCR
—	Not Affected or Applicable.
Stack Pointers	
ISP	Supervisor/Interrupt Stack Pointer
MSP	Supervisor/Master Stack Pointer
SP	Active Stack Pointer
SSP	Supervisor (Master or Interrupt) Stack Pointer
USP	User Stack Pointer
Miscellaneous	
<ea>	Effective Address
<label>	Assemble Program Label
<list>	List of registers, for example D3–D0.
LB	Lower Bound
m	Bit m of an Operand
m–n	Bits m through n of Operand
UB	Upper Bound

3.1.1 Data Movement Instructions

The MOVE and FMOVE instructions with their associated addressing modes are the basic means of transferring and storing addresses and data. MOVE instructions transfer byte, word, and long-word operands from memory to memory, memory to register, register to memory, and register to register. MOVE instructions transfer word and long-word operands and ensure that only valid address manipulations are executed. In addition to the general MOVE instructions, there are several special data movement instructions: MOVE16, MOVEM, MOVEP, MOVEQ, EXG, LEA, PEA, LINK, and UNLK. The MOVE16 instruction is an MC68040 extension to the M68000 instruction set.

The FMOVE instructions move operands into, out of, and between floating-point data registers. FMOVE also moves operands to and from the floating-point control register (FPCR), floating-point status register (FPSR), and floating-point instruction address register (FPIAR). For operands moved into a floating-point data register, FSMOVE and FDMOVE explicitly select single- and double-precision rounding of the result, respectively. FMOVEM moves any combination of either floating-point data registers or floating-point control registers. Table 3-2 lists the general format of these integer and floating-point data movement instructions.

Table 3-2. Data Movement Operation Format

Instruction	Operand Syntax	Operand Size	Operation
EXG	Rn, Rn	32	$Rn \leftarrow \rightarrow Rn$
FMOVE	FPm,FPn <ea>,FPn FPm,<ea> <ea>,FPcr FPcr,<ea>	X B, W, L, S, D, X, P B, W, L, S, D, X, P 32 32	Source $\rightarrow$ Destination
FSMOVE, FDMOVE	FPm,FPn <ea>,FPn	X B, W, L, S, D, X	Source $\rightarrow$ Destination; round destination to single or double precision.
FMOVEM	<ea>,<list> ¹ <ea>,Dn <list> ¹ ,<ea> Dn,<ea>	32, X X 32, X X	Listed Registers $\rightarrow$ Destination Source $\rightarrow$ Listed Registers
LEA	<ea>,An	32	<ea> $\rightarrow$ An
LINK	An,<d>	16, 32	SP – 4 $\rightarrow$ SP; An $\rightarrow$ (SP); SP $\rightarrow$ An, SP + D $\rightarrow$ SP
MOVE MOVE16 MOVEA	<ea>,<ea> <ea>,<ea> <ea>,An	8, 16, 32 16 bytes 16, 32 $\rightarrow$ 32	Source $\rightarrow$ Destination Aligned 16-Byte Block $\rightarrow$ Destination
MOVEM	list,<ea> <ea>,list	16, 32 16, 32 $\rightarrow$ 32	Listed Registers $\rightarrow$ Destination Source $\rightarrow$ Listed Registers
MOVEP	Dn, (d ₁₆ ,An) (d ₁₆ ,An),Dn	16, 32	Dn 31–24 $\rightarrow$ (An + d _n); Dn 23–16 $\rightarrow$ (An + d _n + 2); Dn 15–8 $\rightarrow$ (An + d _n + 4); Dn 7–0 $\rightarrow$ (An + d _n + 6) (An + d _n) $\rightarrow$ Dn 31–24; (An + d _n + 2) $\rightarrow$ Dn 23–16; (An + d _n + 4) $\rightarrow$ Dn 15–8; (An + d _n + 6) $\rightarrow$ Dn 7–0
MOVEQ	#<data>,Dn	8 $\rightarrow$ 32	Immediate Data $\rightarrow$ Destination
PEA	<ea>	32	SP – 4 $\rightarrow$ SP; <ea> $\rightarrow$ (SP)
UNLK	An	32	An $\rightarrow$ SP; (SP) $\rightarrow$ An; SP + 4 $\rightarrow$ SP

NOTE: A register list includes any combination of the eight floating-point data registers or any combination of three control registers (FPCR, FPSR, and FPIAR). If a register list mask resides in a data register, only floating-point data registers may be specified.

3.1.2 Integer Arithmetic Instructions

The integer arithmetic operations include four basic operations: ADD, SUB, MUL, and DIV. They also include CMP, CMPM, CMP2, CLR, and NEG. The instruction set includes ADD, CMP, and SUB instructions for both address and data operations with all operand sizes valid for data operations. Address operands consist of 16 or 32 bits. The CLR and NEG instructions apply to all sizes of data operands. Signed and unsigned MUL and DIV instructions include:

- Word multiply to produce a long-word product.
- Long-word multiply to produce a long-word or quad-word product.
- Long word divided by a word divisor (word quotient and word remainder).
- Long word or quad word divided by a long-word divisor (long-word quotient and long-word remainder).

A set of extended instructions provides multiprecision and mixed-size arithmetic: ADDX, SUBX, EXT, and NEGX. Refer to Table 3-3 for a summary of the integer arithmetic operations. In Table 3-3, X refers to the X-bit in the CCR.

Table 3-3. Integer Arithmetic Operation Format

Instruction	Operand Syntax	Operand Size	Operation
ADD	Dn,<ea>	8, 16, 32	Source + Destination → Destination
ADDA	<ea>,Dn <ea>,An	8, 16, 32 16, 32	
ADDI	#<data>,<ea>	8, 16, 32	Immediate Data + Destination → Destination
ADDQ	#<data>,<ea>	8, 16, 32	
ADDX	Dn,Dn –(An), –(An)	8, 16, 32 8, 16, 32	Source + Destination + X → Destination
CLR	<ea>	8, 16, 32	0 → Destination
CMP	<ea>,Dn	8, 16, 32	Destination – Source
CMPA	<ea>,An	16, 32	
CMPI	#<data>,<ea>	8, 16, 32	Destination – Immediate Data
CMPM	(An)+,(An)+	8, 16, 32	Destination – Source
CMP2	<ea>,Rn	8, 16, 32	Lower Bound → Rn → Upper Bound
DIVS/DIVU	<ea>,Dn <ea>,Dr–Dq <ea>,Dq	32 ÷ 16 → 16,16 64 ÷ 32 → 32,32 32 ÷ 32 → 32	Destination ÷ Source → Destination (Signed or Unsigned Quotient, Remainder)
DIVSL/DIVUL	<ea>,Dr–Dq	32 ÷ 32 → 32,32	
EXT	Dn	8 → 16	Sign-Extended Destination → Destination
EXTB	Dn Dn	16 → 32 8 → 32	
MULS/MULU	<ea>,Dn <ea>,DI <ea>,Dh–DI	16 x 16 → 32 32 x 32 → 32 32 x 32 → 64	Source x Destination → Destination (Signed or Unsigned)
NEG	<ea>	8, 16, 32	0 – Destination → Destination
NEGX	<ea>	8, 16, 32	0 – Destination – X → Destination
SUB	<ea>,Dn	8, 16, 32	Destination = Source → Destination
SUBA	Dn,<ea> <ea>,An	8, 16, 32 16, 32	
SUBI	#<data>,<ea>	8, 16, 32	Destination – Immediate Data → Destination
SUBQ	#<data>,<ea>	8, 16, 32	
SUBX	Dn,Dn –(An), –(An)	8, 16, 32 8, 16, 32	Destination – Source – X → Destination

3.1.3 Logical Instructions

The logical operation instructions (AND, OR, EOR, and NOT) perform logical operations with all sizes of integer data operands. A similar set of immediate instructions (ANDI, ORI, and EORI) provides these logical operations with all sizes of immediate data. Table 3-4 summarizes the logical operations.

Table 3-4. Logical Operation Format

Instruction	Operand Syntax	Operand Size	Operation
AND	<ea>,Dn Dn,<ea>	8, 16, 32 8, 16, 32	Source $\wedge$ Destination $\rightarrow$ Destination
ANDI	#<data>,<ea>	8, 16, 32	Immediate Data $\wedge$ Destination $\rightarrow$ Destination
EOR	Dn,<ea>	8, 16, 32	Source $\oplus$ Destination $\rightarrow$ Destination
EORI	#<data>,<ea>	8, 16, 32	Immediate Data $\oplus$ Destination $\rightarrow$ Destination
NOT	<ea>	8, 16, 32	$\sim$ Destination $\rightarrow$ Destination
OR	<ea>,Dn Dn,<ea>	8, 16, 32	Source $\vee$ Destination $\rightarrow$ Destination
ORI	#<data>,<ea>	8, 16, 32	Immediate Data $\vee$ Destination $\rightarrow$ Destination

3.1.4 Shift and Rotate Instructions

The ASR, ASL, LSR, and LSL instructions provide shift operations in both directions. The ROR, ROL, ROXR, and ROXL instructions perform rotate (circular shift) operations, with and without the CCR extend bit (X-bit). All shift and rotate operations can be performed on either registers or memory.

Register shift and rotate operations shift all operand sizes. The shift count can be specified in the instruction operation word (to shift from 1 – 8 places) or in a register (modulo 64 shift count).

Memory shift and rotate operations shift word operands one bit position only. The SWAP instruction exchanges the 16-bit halves of a register. Fast byte swapping is possible by using the ROR and ROL instructions with a shift count of eight, enhancing the performance of the shift/rotate instructions. Table 3-5 is a summary of the shift and rotate operations. In Table 3-5, C and X refer to the C-bit and X-bit in the CCR.

Table 3-5. Shift and Rotate Operation Format

Instruction	Operand Syntax	Operand Size	Operation
ASL	Dn, Dn # <data>, Dn ea	8, 16, 32 8, 16, 32 16	
ASR	Dn, Dn # <data>, Dn ea	8, 16, 32 8, 16, 32 16	
LSL	Dn, Dn # <data>, Dn ea	8, 16, 32 8, 16, 32 16	
LSR	Dn, Dn # <data>, Dn ea	8, 16, 32 8, 16, 32 16	
ROL	Dn, Dn # <data>, Dn ea	8, 16, 32 8, 16, 32 16	
ROR	Dn, Dn # <data>, Dn ea	8, 16, 32 8, 16, 32 16	
ROXL	Dn, Dn # <data>, Dn ea	8, 16, 32 8, 16, 32 16	
ROXR	Dn, Dn # <data>, Dn ea	8, 16, 32 8, 16, 32 16	
SWAP	Dn	32	

NOTE: X indicates the extend bit and C the carry bit in the CCR.

3.1.5 Bit Manipulation Instructions

BTST, BSET, BCLR, and BCHG are bit manipulation instructions. All bit manipulation operations can be performed on either registers or memory. The bit number is specified either as immediate data or in the contents of a data register. Register operands are 32 bits long, and memory operands are 8 bits long. Table 3-6 summarizes bit manipulation operations; Z refers to the zero bit of the CCR.

Table 3-6. Bit Manipulation Operation Format

Instruction	Operand Syntax	Operand Size	Operation
BCHG	Dn,<ea> #<data>,<ea>	8, 32 8, 32	~ (<Bit Number> of Destination) → Z → Bit of Destination
BCLR	Dn,<ea> #<data>,<ea>	8, 32 8, 32	~ (<Bit Number> of Destination) → Z; 0 → Bit of Destination
BSET	Dn,<ea> #<data>,<ea>	8, 32 8, 32	~ (<Bit Number> of Destination) → Z; 1 → Bit of Destination
BTST	Dn,<ea> #<data>,<ea>	8, 32 8, 32	~ (<Bit Number> of Destination) → Z

3.1.6 Bit Field Instructions

The M68000 family architecture supports variable-length bit field operations on fields of up to 32 bits. The BFINS instruction inserts a value into a bit field. BFEXTU and BFEXTS extract a value from the field. BFFFO finds the first set bit in a bit field. Also included are instructions analogous to the bit manipulation operations: BFTST, BFSET, BFCLR, and BFCHG. Table 3-7 summarizes bit field operations.

Table 3-7. Bit Field Operation Format

Instruction	Operand Syntax	Operand Size	Operation
BFCHG	<ea> {offset:width}	1–32	~ Field → Field
BFCLR	<ea> {offset:width}	1–32	0's → Field
BFEXTS	<ea> {offset:width}, Dn	1–32	Field → Dn; Sign-Extended
BFEXTU	<ea> {offset:width}, Dn	1–32	Field → Dn; Zero-Extended
BFFFO	<ea> {offset:width}, Dn	1–32	Scan for First Bit Set in Field; Offset → Dn.
BFINS	Dn,<ea> {offset:width}	1–32	Dn → Field
BFSET	<ea> {offset:width}	1–32	1's → Field
BFTST	<ea> {offset:width}	1–32	Field MSB → N; ~ (OR of All Bits in Field) → Z

NOTE: All bit field instructions set the CCR N and Z bits as shown for BFTST before performing the specified operation.

3.1.7 Binary-Coded Decimal Instructions

Five instructions support operations on binary-coded decimal (BCD) numbers. The arithmetic operations on packed BCD numbers are ABCD, SBCD, and NBCD. PACK and UNPK instructions aid in the conversion of byte-encoded numeric data, such as ASCII or EBCDIC strings to BCD data and vice versa. Table 3-8 summarizes BCD operations. In Table 3-8 X refers to the X-bit in the CCR.

Table 3-8. Binary-Coded Decimal Operation Format

Instruction	Operand Syntax	Operand Size	Operation
ABCD	Dn,Dn -(An), -(An)	8 8	Source ₁₀ + Destination ₁₀ + X → Destination
NBCD	<ea>	8	0 – Destination ₁₀ – X → Destination
PACK	-(An), -(An) #<data> Dn,Dn,#<data>	16 → 8 16 → 8	Unpackaged Source + Immediate Data → Packed Destination
SBCD	Dn,Dn -(An), -(An)	8 8	Destination ₁₀ – Source ₁₀ – X → Destination
UNPK	-(An),-(An) #<data> Dn,Dn,#<data>	8 → 16 8 → 16	Packed Source → Unpacked Source Unpacked Source + Immediate Data → Unpacked Destination

3.1.8 Program Control Instructions

A set of subroutine call and return instructions and conditional and unconditional branch instructions perform program control operations. Also included are test operand instructions (TST and FTST), which set the integer or floating-point condition codes for use by other program and system control instructions. NOP forces synchronization of the internal pipelines. Table 3-9 summarizes these instructions.

Table 3-9. Program Control Operation Format

Instruction	Operand Syntax	Operand Size	Operation
Integer and Floating-Point Conditional			
Bcc, FBcc	<label>	8, 16, 32	If Condition True, Then $PC + d_n \rightarrow PC$
DBcc, FDBcc	Dn,<label>	16	If Condition False, Then $Dn - 1 \rightarrow Dn$ If $Dn \rightarrow -1$, Then $PC + d_n \rightarrow PC$
Scc, FScc	<ea>	8	If Condition True, Then 1's $\rightarrow$ Destination; Else 0's $\rightarrow$ Destination
Unconditional			
BRA	<label>	8, 16, 32	$PC + d_n \rightarrow PC$
BSR	<label>	8, 16, 32	$SP - 4 \rightarrow SP$; $PC \rightarrow (SP)$; $PC + d_n \rightarrow PC$
JMP	<ea>	none	Destination $\rightarrow PC$
JSR	<ea>	none	$SP - 4 \rightarrow SP$; $PC \rightarrow (SP)$; Destination $\rightarrow PC$
NOP	none	none	$PC + 2 \rightarrow PC$ (Integer Pipeline Synchronized)
FNOP	none	none	$PC + 4 \rightarrow PC$ (FPU Pipeline Synchronized)
Returns			
RTD	#<data>	16	$(SP) \rightarrow PC$; $SP + 4 + d_n \rightarrow SP$
RTR	none	none	$(SP) \rightarrow CCR$; $SP + 2 \rightarrow SP$; $(SP) \rightarrow PC$; $SP + 4 \rightarrow SP$
RTS	none	none	$(SP) \rightarrow PC$; $SP + 4 \rightarrow SP$
Test Operand			
TST	<ea>	8, 16, 32	Set Integer Condition Codes
FTST	<ea> FPn	B, W, L, S, D, X, P X	Set Floating-Point Condition Codes

Letters cc in the integer instruction mnemonics Bcc, DBcc, and Scc specify testing one of the following conditions:

CC—Carry clear	GE—Greater than or equal
LS—Lower or same	PL—Plus
CS—Carry set	GT—Greater than
LT—Less than	T—Always true*
EQ—Equal	HI—Higher
MI—Minus	VC—Overflow clear
F—Never true*	LE—Less than or equal
NE—Not equal	VS—Overflow set

*Not applicable to the Bcc instructions.

3.1.9 System Control Instructions

Privileged and trapping instructions as well as instructions that use or modify the CCR provide system control operations. FSAVE and FRESTORE save and restore the nonuser visible portion of the FPU during context switches in a virtual memory or multitasking system. The conditional trap instructions, which use the same conditional tests as their corresponding program control instructions, allow an optional 16- or 32-bit immediate operand to be included as part of the instruction for passing parameters to the operating system. These instructions cause the processor to flush the instruction pipe. Table 3-10 summarizes these instructions. See 3.2 Integer Unit Condition Code Computation for more details on condition codes.

Table 3-10. System Control Operation Format

Instruction	Operand Syntax	Operand Size	Operation
Privileged			
ANDI to SR	#<data>,SR	16	Immediate Data $\wedge$ SR $\rightarrow$ SR
EORI to SR	#<data>,SR	16	Immediate Data $\oplus$ SR $\rightarrow$ SR
FRESTORE	<ea>	none	State Frame $\rightarrow$ Internal Floating-Point Registers
FSAVE	<ea>	none	Internal Floating-Point Registers $\rightarrow$ State Frame
MOVE to SR	<ea>,SR	16	Source $\rightarrow$ SR
MOVE from SR	SR,<ea>	16	SR $\rightarrow$ Destination
MOVE USP	USP,An	32	USP $\rightarrow$ An
	An,USP	32	An $\rightarrow$ USP
MOVEC	Rc,Rn	32	Rc $\rightarrow$ Rn
	Rn,Rc	32	Rn $\rightarrow$ Rc
MOVES	Rn,<ea>	8, 16, 32	Rn $\rightarrow$ Destination Using DFC
	<ea>,Rn		Source Using SFC $\rightarrow$ Rn
ORI to SR	#<data>,SR	16	Immediate Data $\vee$ SR $\rightarrow$ SR
RESET	none	none	Assert Reset Output
RTE	none	none	(SP) $\rightarrow$ SR; SP + 2 $\rightarrow$ SP; (SP) $\rightarrow$ PC; SP + 4 $\rightarrow$ SP; Restore Stack According to Format
STOP	#<data>	16	Immediate Data $\rightarrow$ SR; STOP
Trap Generating			
BKPT	#<data>	none	Run Breakpoint Cycle
CHK	<ea>,Dn	16, 32	If Dn < 0 or Dn > (<ea>), Then CHK Exception
CHK2	<ea>,Rn	8, 16, 32	If Rn < Lower Bound or Rn > Upper Bound, Then CHK Exception
ILLEGAL	none	none	SSP - 2 $\rightarrow$ SSP; Vector Offset $\rightarrow$ (SSP); SSP - 4 $\rightarrow$ SSP; PC $\rightarrow$ (SSP); SSP - 2 $\rightarrow$ SSP; SR $\rightarrow$ (SSP); Illegal Instruction Vector Address $\rightarrow$ PC
TRAP	#<data>	none	SSP - 2 $\rightarrow$ SSP; Format and Vector Offset $\rightarrow$ (SSP) SSP - 4 $\rightarrow$ SSP; PC $\rightarrow$ (SSP); SSP - 2 $\rightarrow$ SSP; SR $\rightarrow$ (SSP); Vector Address $\rightarrow$ PC
TRAPcc	none #<data>	none 16, 32	If cc True, Then Trap Exception
FTRAPcc	none #<data>	none 16, 32	If Floating-Point cc True, Then Trap Exception
TRAPV	none	none	If V, Then Take Overflow Trap Exception
Condition Code Register			
ANDI to SR	#<data>,CCR	8	Immediate Data $\wedge$ CCR $\rightarrow$ CCR
EORI to SR	#<data>,CCR	8	Immediate Data $\oplus$ CCR $\rightarrow$ CCR
MOVE to SR	<ea>,CCR	16	Source $\rightarrow$ CCR
MOVE from SR	CCR,<ea>	16	CCR $\rightarrow$ Destination
ORI to SR	#<data>,CCR	8	Immediate Data $\vee$ CCR $\rightarrow$ CCR

Letters cc in the TRAPcc and FTRAPcc specify testing for a condition.

3.1.10 Cache Control Instructions (MC68040)

The cache instructions provide maintenance functions for managing the instruction and data caches. CINV invalidates cache entries in both caches, and CPUSH pushes dirty data from the data cache to update memory. Both instructions can operate on either or both caches and can select a single cache line, all lines in a page, or the entire cache. Table 3-11 summarizes these instructions.

Table 3-11. Cache Control Operation Format

Instruction	Operand Syntax	Operand Size	Operation
CINVL	caches,(An)	none	Invalidate cache line
CINVP	caches, (An)	none	Invalidate cache page
CINVA	caches	none	Invalidate entire cache
CPUSHL CPUSHP CPUSHA	caches,(An) caches, (An) caches	none none none	Push selected dirty data cache lines, then invalidate selected cache lines

3.1.11 Multiprocessor Instructions

The TAS, CAS, and CAS2 instructions coordinate the operations of processors in multiprocessing systems. These instructions use read- modify-write bus cycles to ensure uninterrupted updating of memory. Coprocessor instructions control the coprocessor operations. Table 3- 12 summarizes these instructions.

Table 3-12. Multiprocessor Operations

Instruction	Operand Syntax	Operand Size	Operation
Read-Write-Modify			
CAS	Dc,Du,<ea>	8, 16, 32	Destination – Dc → CC If Z, Then Du → Destination Else Destination → Dc
CAS2	Dc1–Dc2, Du1–Du2, (Rn)–(Rn)	16, 32	Dual Operand CAS
TAS	<ea>	8	Destination – 0; Set Condition Codes; 1 → Destination [7]
Coprocessor			
cpBcc	<label>	16, 32	If cpcc True, Then PC + d _n → PC
cpDBcc	<label>,Dn	16	If cpcc False, Then Dn – 1 → Dn If Dn ≠ –1, Then PC + d _n → PC
cpGEN	User Defined	User Defined	Operand → Coprocessor
cpRESTORE	<ea>	none	Restore Coprocessor State from <ea>
cpSAVE	<ea>	none	Save Coprocessor State at <ea>
cpScc	<ea>	8	If cpcc True, Then 1's → Destination; Else 0's → Destination
cpTRAPcc	none #<data>	none 16, 32	If cpcc True, Then TRAPcc Exception

3.1.12 Memory Management Unit (MMU) Instructions

The PFLUSH instructions flush the address translation caches (ATCs) and can optionally select only nonglobal entries for flushing. PTEST performs a search of the address translation tables, stores the results in the MMU status register, and loads the entry into the ATC. Table 3-13 summarizes these instructions.

Table 3-13. MMU Operation Format

Instruction	Processor	Operand Syntax	Operand Size	Operation
PBcc	MC68851	<label>	none	Branch on PMMU Condition
PDBcc	MC68851	Dn,<label>	none	Test, Decrement, and Branch
PFLUSHA	MC68030 MC68040 MC68851	none	none	Invalidate All ATC Entries
PFLUSH	MC68040	(An)	none	Invalidate ATC Entries at Effective Address
PFLUSHN	MC68040	(An)	none	Invalidate Nonglobal ATC Entries at Effective Address
PFLUSHAN	MC68040	none	none	Invalidate All Nonglobal ATC Entries
PFLUSHS	MC68851	none	none	Invalidate All Shared/Global ATC Entries
PFLUSHR	MC68851	<ea>	none	Invalidate ATC and RPT Entries
PLOAD	MC68030 MC68851	FC,<ea>	none	Load an Entry into the ATC
PMOVE	MC68030 MC68851	MRn,<ea> <ea>,MRn	8,16,32,64	Move to/from MMU Registers
PRESTORE	MC68851	<ea>	none	PMMU Restore Function
PSAVE	MC68851	<ea>	none	PMMU Save Function
PScC	MC68851	<ea>	8	Set on PMMU Condition
PTEST	MC68030 MC68040 MC68851	(An)	none	Information About Logical Address → MMU Status Register
PTRAPcc	MC68851	#<data>	16,32	Trap on PMMU Condition

3.1.13 Floating-Point Arithmetic Instructions

The following paragraphs describe the floating-point instructions, organized into two categories of operation: dyadic (requiring two operands) and monadic (requiring one operand).

The dyadic floating-point instructions provide several arithmetic functions that require two input operands, such as add and subtract. For these operations, the first operand can be located in memory, an integer data register, or a floating-point data register. The second operand is always located in a floating-point data register. The results of the operation store in the register specified as the second operand. All FPU operations support all data formats. Results are rounded to either extended-, single-, or double-precision format. Table 3-14 gives the general format of dyadic instructions, and Table 3-15 lists the available operations.

Table 3-14. Dyadic Floating-Point Operation Format

Instruction	Operand Syntax	Operand Format	Operation
F<dop>	<ea>,FPn FPm,FPn	B, W, L, S, D, X, P X	FPn <Function> Source → FPn

NOTE: < dop > is any one of the dyadic operation specifiers.

Table 3-15. Dyadic Floating-Point Operations

Instruction	Operation
FADD, FSADD, FDADD	Add
FCMP	Compare
FDIV, FSDIV, FDDIV	Divide
FMOD	Modulo Remainder
FMUL, FSMUL, FDMUL	Multiply
FREM	IEEE Remainder
FSCALE	Scale Exponent
FSUB, FSSUB, FDSUB	Subtract
FSGLDIV, FSGLMUL	Single-Precision Divide, Multiply

The monadic floating-point instructions provide several arithmetic functions requiring only one input operand. Unlike the integer counterparts to these functions (e.g., NEG < ea >), a source and a destination can be specified. The operation is performed on the source operand and the result is stored in the destination, which is always a floating-point data register. When the source is not a floating-point data register, all data formats are supported. The data format is always extended precision for register-to-register operations. Table 3-16 lists the general format of these instructions, and Table 3-17 lists the available operations.

Table 3-16. Monadic Floating-Point Operation Format

Instruction	Operand Syntax	Operand Format	Operation
F<mop>	<ea>,FPn FPm,FPn FPn	B, W, L, S, D, X, P X X	Source → Function → FPn FPn → Function → FPn

NOTE: < mop > is any one of the monadic operation specifiers.

Table 3-17. Monadic Floating-Point Operations

Instruction	Operation	Instruction	Operation
FABS	Absolute Value	FLOGN	$\ln(x)$
FACOS	Arc Cosine	FLOGNP1	$\ln(x + 1)$
FASIN	Arc Sine	FLOG10	$\log_{10}(x)$
FATAN	Hyperbolic Art Tangent	FLOG2	$\log_2(x)$
FCOS	Cosine	FNEG	Negate
FCOSH	Hyperbolic Cosine	FSIN	Sine
FETOX	e^x	FSINH	Hyperbolic Sine
FETOXM1	$e^x - 1$	FSQRT	Square Root
FGETEXP	Extract Exponent	FTAN	Tangent
FGETMAN	Extract Mantissa	FTANH	Hyperbolic Tangent
FINT	Extract Integer Part	FTENTOX	10^x
FINTRZ	Extract Integer Part, Rounded-to-Zero	FTWOTOX	2^x

3.2 INTEGER UNIT CONDITION CODE COMPUTATION

Many integer instructions affect the CCR to indicate the instruction's results. Program and system control instructions also use certain combinations of these bits to control program and system flow. The condition codes meet consistency criteria across instructions, uses, and instances. They also meet the criteria of meaningful results, where no change occurs unless it provides useful information. Refer to **Section 1 Introduction** for details concerning the CCR.

Table 3-18 lists the integer condition code computations for instructions and Table 3-19 lists the condition names, encodings, and tests for the conditional branch and set instructions. The test associated with each condition is a logical formula using the current states of the condition codes. If this formula evaluates to one, the condition is true. If the formula evaluates to zero, the condition is false. For example, the T condition is always true, and the EQ condition is true only if the Z-bit condition code is currently true.

Table 3-18. Integer Unit Condition Code Computations

Operations	X	N	Z	V	C	Special Definition
ABCD	*	U	?	U	?	C = Decimal Carry $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
ADD, ADDI, ADDQ	*	*	*	?	?	$V = Sm \wedge Dm \wedge \overline{Rm} \vee \overline{Sm} \wedge \overline{Dm} \wedge Rm$ $C = Sm \wedge Dm \vee \overline{Rm} \wedge Dm \vee Sm \wedge \overline{Rm}$
ADDX	*	*	?	?	?	$V = Sm \wedge Dm \wedge \overline{Rm} \vee \overline{Sm} \wedge \overline{Dm} \wedge Rm$ $C = Sm \wedge Dm \vee \overline{Rm} \wedge Dm \vee Sm \wedge \overline{Rm}$ $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
AND, ANDI, EOR, EORI, MOVEQ, MOVE, OR, ORI, CLR, EXT, EXTB, NOT, TAS, TST	—	*	*	0	0	
CHK	—	*	U	U	U	
CHK2, CMP2	—	U	?	U	?	$Z = (R = LB) \vee (R = UB)$ $C = (LB \leq UB) \wedge (IR < LB) \vee (R > UB)$ $V = (UB < LB) \wedge (R > UB) \wedge (R < LB)$
SUB, SUBI, SUBQ	*	*	*	?	?	$V = \overline{Sm} \wedge Dm \wedge \overline{Rm} \vee Sm \wedge \overline{Dm} \wedge Rm$ $C = Sm \wedge \overline{Dm} \vee Rm \wedge \overline{Dm} \vee Sm \wedge Rm$
SUBX	*	*	?	?	?	$V = \overline{Sm} \wedge Dm \wedge \overline{Rm} \vee Sm \wedge \overline{Dm} \wedge Rm$ $C = Sm \wedge \overline{Dm} \vee Rm \wedge \overline{Dm} \vee Sm \wedge Rm$ $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
CAS, CAS2, CMP, CMPA, CMPI, CMPM	—	*	*	?	?	$V = \overline{Sm} \wedge Dm \wedge \overline{Rm} \vee Sm \wedge \overline{Dm} \wedge Rm$ $C = Sm \wedge \overline{Dm} \vee Rm \wedge \overline{Dm} \vee Sm \wedge Rm$
DIVS, DUVU	—	*	*	?	0	V = Division Overflow
MULS, MULU	—	*	*	?	0	V = Multiplication Overflow
SBCD, NBCD	*	U	?	U	?	C = Decimal Borrow $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
NEG	*	*	*	?	?	$V = Dm \wedge Rm$ $C = Dm \vee Rm$
NEGX	*	*	?	?	?	$V = Dm \wedge Rm$ $C = Dm \vee Rm$ $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
BTST, BCHG, BSET, BCLR	—	—	?	—	—	$Z = \overline{Dn}$
BFTST, BFCHG, BFSET, BFCLR	—	?	?	0	0	$N = Dm$ $Z = \overline{Dn} \wedge \overline{Dm-1} \wedge \dots \wedge \overline{D0}$
BFEXTS, BFEXTU, BFFFO	—	?	?	0	0	$N = Sm$ $Z = Sm \wedge \overline{Sm-1} \wedge \dots \wedge \overline{S0}$
BFINS	—	?	?	0	0	$N = Dm$ $Z = \overline{Dm} \wedge \overline{Dm-1} \wedge \dots \wedge \overline{D0}$
ASL	*	*	*	?	?	$V = Dm \wedge \overline{Dm-1} \vee \dots \vee \overline{Dm-r} \vee \overline{Dm} \wedge$ $(Dm-1 \vee \dots \vee Dm-r)$ $C = \overline{Dm-r+1}$
ASL (r = 0)	—	*	*	0	0	
LSL, ROXL	*	*	*	0	?	$C = Dm - r + 1$

Table 3-18. Integer Unit Condition Code Computations (Continued)

Operations	X	N	Z	V	C	Special Definition
LSR (r = 0)	—	*	*	0	0	
ROXL (r = 0)	—	*	*	0	?	X = C
ROL	—	*	*	0	?	C = Dm – r + 1
ROL (r = 0)	—	*	*	0	0	
ASR, LSR, ROXR	*	*	*	0	?	C = Dr – 1
ASR, LSR (r = 0)	—	*	*	0	0	
ROXR (r = 0)	—	*	*	0	?	X = C
ROR	—	*	*	0	?	C = Dr – 1
ROR (r = 0)	—	*	*	0	0	

? = Other—See Special Definition

N = Result Operand (MSB)

 $Z = \overline{Rm} \wedge \dots \wedge \overline{R0}$

Sm = Source Operand (MSB)

Dm = Destination Operand (MSB)

Rm = Result Operand (MSB)

 $\overline{Rm}$ = Not Result Operand (MSB)

R = Register Tested

r = Shift Count

Table 3-19. Conditional Tests

Mnemonic	Condition	Encoding	Test
T*	True	0000	1
F*	False	0001	0
HI	High	0010	$\overline{C} \wedge \overline{Z}$
LS	Low or Same	0011	$C \vee Z$
CC(HI)	Carry Clear	0100	C
CS(LO)	Carry Set	0101	C
NE	Not Equal	0110	Z
EQ	Equal	0111	Z
VC	Overflow Clear	1000	V
VS	Overflow Set	1001	V
PL	Plus	1010	N
MI	Minus	1011	N
GE	Greater or Equal	1100	$N \wedge V \vee \overline{N} \wedge \overline{V}$
LT	Less Than	1101	$N \wedge \overline{V} \vee \overline{N} \wedge V$
GT	Greater Than	1110	$N \wedge V \wedge \overline{Z} \vee \overline{N} \wedge \overline{V} \wedge \overline{Z}$
LE	Less or Equal	1111	$Z \vee N \wedge \overline{V} \vee \overline{N} \wedge V$

NOTES:
 $\overline{N}$ = Logical Not N

 $\overline{V}$ = Logical Not V

 $\overline{Z}$ = Logical Not Z

*Not available for the Bcc instruction.

3.3 INSTRUCTION EXAMPLES

The following paragraphs provide examples of how to use selected instructions.

3.3.1 Using the Cas and Cas2 Instructions

The CAS instruction compares the value in a memory location with the value in a data register, and copies a second data register into the memory location if the compared values are equal. This provides a means of updating system counters, history information, and globally shared pointers. The instruction uses an indivisible read-modify- write cycle. After CAS reads the memory location, no other instruction can change that location before CAS has written the new value. This provides security in single-processor systems, in multitasking environments, and in multiprocessor environments. In a single-processor system, the operation is protected from instructions of an interrupt routine. In a multitasking environment, no other task can interfere with writing the new value of a system variable. In a multiprocessor environment, the other processors must wait until the CAS instruction completes before accessing a global pointer.

3.3.2 Using the Moves Instruction

This instruction moves the byte, word, or long-word operand from the specified general register to a location within the address space specified by the destination function code (DFC) register. It also moves the byte, word, or long-word operand from a location within the address space specified by the source function code (SFC) register to the specified general register.

3.3.3 Nested Subroutine Calls

The LINK instruction pushes an address onto the stack, saves the stack address at which the address is stored, and reserves an area of the stack. Using this instruction in a series of subroutine calls results in a linked list of stack frames.

The UNLK instruction removes a stack frame from the end of the list by loading an address into the stack pointer and pulling the value at that address from the stack. When the operand of the instruction is the address of the link address at the bottom of a stack frame, the effect is to remove the stack frame from the stack and from the linked list.

3.3.4 Bit Field Instructions

One of the data types provided by the MC68030 is the bit field, consisting of as many as 32 consecutive bits. An offset from an effective address and a width value defines a bit field. The offset is a value in the range of -231 through $231 - 1$ from the most significant bit (bit 7) at the effective address. The width is a positive number, 1 through 32. The most significant bit of a bit field is bit 0. The bits number in a direction opposite to the bits of an integer.

The instruction set includes eight instructions that have bit field operands. The insert bit field (BFINS) instruction inserts a bit field stored in a register into a bit field. The extract bit field signed (BFEXTS) instruction loads a bit field into the least significant bits of a register and

extends the sign to the left, filling the register. The extract bit field unsigned (BFEXTU) also loads a bit field, but zero fills the unused portion of the destination register.

The set bit field (BFSET) instruction sets all the bits of a field to ones. The clear bit field (BFCLR) instruction clears a field. The change bit field (BFCHG) instruction complements all the bits in a bit field. These three instructions all test the previous value of the bit field, setting the condition codes accordingly. The test bit field (BFTST) instruction tests the value in the field, setting the condition codes appropriately without altering the bit field. The find first one in bit field (BFFFO) instruction scans a bit field from bit 0 to the right until it finds a bit set to one and loads the bit offset of the first set bit into the specified data register. If no bits in the field are set, the field offset and the field width is loaded into the register.

An important application of bit field instructions is the manipulation of the exponent field in a floating-point number. In the IEEE standard format, the most significant bit is the sign bit of the mantissa. The exponent value begins at the next most significant bit position; the exponent field does not begin on a byte boundary. The extract bit field (BFEXTU) instruction and the BFTST instruction are the most useful for this application, but other bit field instructions can also be used.

Programming of input and output operations to peripherals requires testing, setting, and inserting of bit fields in the control registers of the peripherals. This is another application for bit field instructions. However, control register locations are not memory locations; therefore, it is not always possible to insert or extract bit fields of a register without affecting other fields within the register.

Another widely used application for bit field instructions is bit- mapped graphics. Because byte boundaries are ignored in these areas of memory, the field definitions used with bit field instructions are very helpful.

3.3.5 Pipeline Synchronization with the Nop Instruction

Although the no operation (NOP) instruction performs no visible operation, it serves an important purpose. It forces synchronization of the integer unit pipeline by waiting for all pending bus cycles to complete. All previous integer instructions and floating-point external operand accesses complete execution before the NOP begins. The NOP instruction does not synchronize the FPU pipeline—floating- point instructions with floating-point register operand destinations can be executing when the NOP begins. NOP is considered a change of flow instruction and traps for trace on change of flow. A single- cycle nonsynchronizing operation can be affected with the TRAPF instruction.

3.4 FLOATING-POINT INSTRUCTION DETAILS

The following paragraphs describe the operation tables used in the instruction descriptions and the conditional tests that can be used to change program flow based on floating-point conditions. Details on NaNs and floating-point condition codes are also discussed. The IEEE 754 standard specifies that each data format must support add, subtract, multiply, divide, remainder, square root, integer part, and compare. In addition to these arithmetic

functions, software supports remainder and integer part; the FPU also supports the nontranscendental operations of absolute value, negate, and test.

Most floating-point instruction descriptions include an operation table. This table lists the resulting data types for the instruction based on the operand,s input. Table 3-20 is an operation table example for the FADD instruction. The operation table lists the source operand type along the top, and the destination operand type along the side. In-range numbers are normalized, denormalized, unnormalized real numbers, or integers that are converted to normalized or denormalized extended-precision numbers upon entering the FPU.

Table 3-20. Operation Table Example (FADD Instruction)

DESTINATION	SOURCE ¹			
	+ In Range	– + Zero –		+ Infinity –
In Range	+	ADD	ADD	+inf –inf
	–			
Zero	+	ADD	+ 0.0 0.0 ²	+inf –inf
	–		0.0 ² –0.0	
Infinity	+	+inf	+inf	+inf NAN ³
	–	–inf	–inf	NAN ³ –inf

NOTES:

- 1.If either operand is a NAN, refer to **1.6.5 NANs** for more information.
- 2.Returns +0.0 in rounding modes RN, RZ, and RP; returns –0.0 in RM.
- 3.Sets the OPERR bit in the FPSR exception byte.

For example, Table 3-20 illustrates that if both the source and destination operand are positive zero, the result is also a positive zero. If the source operand is a positive zero and the destination operand is an in-range number, then the ADD algorithm is executed to obtain the result. If a label such as ADD appears in the table, it indicates that the FPU performs the indicated operation and returns the correct result. Since the result of such an operation is undefined, a NAN is returned as the result, and the OPERR bit is set in the FPSR EXC byte.

In addition to the data types covered in the operation tables for each floating-point instruction, NANs can also be used as inputs to an arithmetic operation. The operation tables do not contain a row and column for NANs because NANs are handled the same way for all operations. If either operand, but not both operands, of an operation is a nonsignaling NAN, then that NAN is returned as the result. If both operands are nonsignaling NANs, then the destination operand nonsignaling NAN is returned as the result.

If either operand to an operation is a signaling NAN (SNAN), then the SNAN bit is set in the FPSR EXC byte. If the SNAN exception enable bit is set in the FPCR ENABLE byte, then the exception is taken and the destination is not modified. If the SNAN exception enable bit is not set, setting the SNAN bit in the operand to a one converts the SNAN to a nonsignaling NAN. The operation then continues as described in the preceding paragraph for nonsignaling NANs.

3.5 FLOATING-POINT COMPUTATIONAL ACCURACY

Representing a real number in a binary format of finite precision is problematic. If the number cannot be represented exactly, a round-off error occurs. Furthermore, when two of these inexact numbers are used in a calculation, the result becomes even more inexact. The IEEE 754 standard defines the error bounds for calculating binary floating-point values so that the result obtained by any conforming device can be predicted exactly for a particular precision and rounding mode. The error bound defined by the IEEE 754 standard is one-half unit in the last place of the destination data format in the RN mode, and one unit in last place in the other rounding modes. The operation's data format must have the same input values, rounding mode, and precision. The standard also specifies the maximum allowable error that can be introduced during a calculation and the manner in which rounding of the result is performed.

The single- and double-precision formats provide emulation for devices that only support those precisions. The execution speed of all instructions is the same whether using single- or double-precision rounding. When using these two data formats, the FPU produces the same results as any other device that conforms to the IEEE standard but does not support extended precision. The results are the same when performing the same operation in extended precision and storing the results in single- or double-precision format.

The FPU performs all floating-point internal operations in extended-precision. It supports mixed-mode arithmetic by converting single- and double-precision operands to extended-precision values before performing the specified operation. The FPU converts all memory data formats to the extended-precision data format and stores the value in a floating-point register or uses it as the source operand for an arithmetic operation. The FPU also converts extended-precision data formats in a floating-point data register to any data format and either stores it in a memory destination or in an integer data register.

Additionally if the external operand is a denormalized number, the number is normalized before an operation is performed. However, an external denormalized number moved into a floating-point data register is stored as a denormalized number. The number is first normalized and then denormalized before it is stored in the designated floating-point data register. This method simplifies the handling of all other data formats and types.

If an external operand is an unnormalized number, the number is normalized before it is used in an arithmetic operation. If the external operand is an unnormalized zero (i.e., with a mantissa of all zeros), the number is converted to a normalized zero before the specified operation is performed. The regular use of unnormalized inputs not only defeats the purpose of the IEEE 754 standard, but also can produce gross inaccuracies in the results.

3.5.1 Intermediate Result

All FPU calculations use an intermediate result. When the FPU performs any operation, the calculation is carried out using extended-precision inputs, and the intermediate result is calculated as if to produce infinite precision. After the calculation is complete, the intermediate result is rounded to the selected precision and stored in the destination.

Figure 3-1 illustrates the intermediate result format. The intermediate result's exponent for some dyadic operations (i.e., multiply and divide) can easily overflow or underflow the 15-bit exponent of the destination floating-point register. To simplify the overflow and underflow detection, intermediate results in the FPU maintain a 16-bit (17 bits for the MC68881 and MC68882), two's complement, integer exponent. Detection of an overflow or underflow intermediate result always converts the 16-bit exponent into a 15-bit biased exponent before being stored in a floating-point data register. The FPU internally maintains the 67-bit mantissa for rounding purposes. The mantissa is always rounded to 64 bits (or less, depending on the selected rounding precision) before it is stored in a floating-point data register.

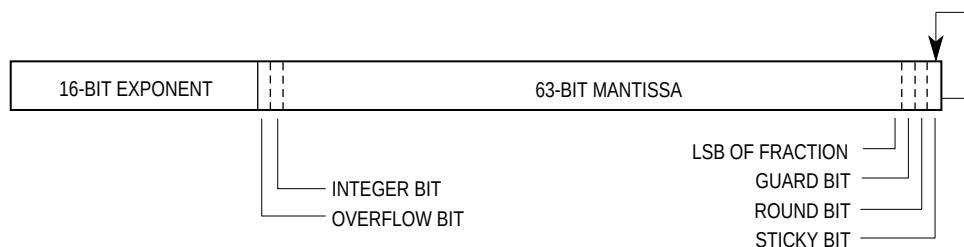


Figure 3-1. Intermediate Result Format

If the destination is a floating-point data register, the result is in the extended-precision format and is rounded to the precision specified by the FPSR PREC bits before being stored. All mantissa bits beyond the selected precision are zero. If the single- or double-precision mode is selected, the exponent value is in the correct range even if it is stored in extended-precision format. If the destination is a memory location, the FPSR PREC bits are ignored. In this case, a number in the extended-precision format is taken from the source floating-point data register, rounded to the destination format precision, and then written to memory.

Depending on the selected rounding mode or destination data format in effect, the location of the least significant bit of the mantissa and the locations of the guard, round, and sticky bits in the 67-bit intermediate result mantissa varies. The guard and round bits are always calculated exactly. The sticky bit is used to create the illusion of an infinitely wide intermediate result. As the arrow illustrates in Figure 3-1, the sticky bit is the logical OR of all the bits in the infinitely precise result to the right of the round bit. During the calculation stage of an arithmetic operation, any non-zero bits generated that are to the right of the round bit set the sticky bit to one. Because of the sticky bit, the rounded intermediate result for all required IEEE arithmetic operations in the RN mode is in error by no more than one half unit in the last place.

3.5.2 Rounding the Result

The FPU supports the four rounding modes specified by the IEEE 754 standard. These modes are round to nearest (RN), round toward zero (RZ), round toward plus infinity (RP), and round toward minus infinity (RM). The RM and RP rounding modes are often referred to as "directed rounding modes" and are useful in interval arithmetic. Rounding is accomplished through the intermediate result. Single-precision results are rounded to a 24-bit boundary; double-precision results are rounded to a 53-bit boundary; and extended-precision results are rounded to a 64-bit boundary. Table 3-21 lists the encodings for the FPCR that denote the rounding and precision modes.

Table 3-21. FPCR Encodings

Rounding Mode (RND Field)	Encoding		Rounding Precision (PREC Field)
To Nearest (RN)	0	0	Extend (X)
To Zero (RZ)	0	1	Single (S)
To Minus Infinity (RM)	1	0	Double (D)
To Plus Infinity (RP)	1	1	Undefined

Rounding the intermediate result's mantissa to the specified precision and checking the 16-bit intermediate exponent to ensure that it is within the representable range of the selected rounding precision accomplishes range control. Range control is a method used to assure correct emulation of a device that only supports single- or double- precision arithmetic. If the intermediate result's exponent exceeds the range of the selected precision, the exponent value appropriate for an underflow or overflow is stored as the result in the 16-bit extended-precision format exponent. For example, if the data format and rounding mode is single precision RM and the result of an arithmetic operation overflows the magnitude of the single-precision format, the largest normalized single-precision value is stored as an extended-precision number in the destination floating-point data register (i.e., an unbiased 15-bit exponent of \$00FF and a mantissa of \$FFFFFF0000000000). If an infinity is the appropriate result for an underflow or overflow, the infinity value for the destination data format is stored as the result (i.e., an exponent with the maximum value and a mantissa of zero).

Figure 3-2 illustrates the algorithm that the FPU uses to round an intermediate result to the selected rounding precision and destination data format. If the destination is a floating-point register, either the selected rounding precision specified by the FPCR PREC status byte or by the instruction itself determines the rounding boundary. For example, FSADD and FDADD specify single- and double-precision rounding regardless of the precision specified in the FPCR PREC status byte. If the destination is external memory or an integer data register, the destination data format determines the rounding boundary. If the rounded result of an operation is not exact, then the INEX2 bit is set in the FPSR EXC status byte.

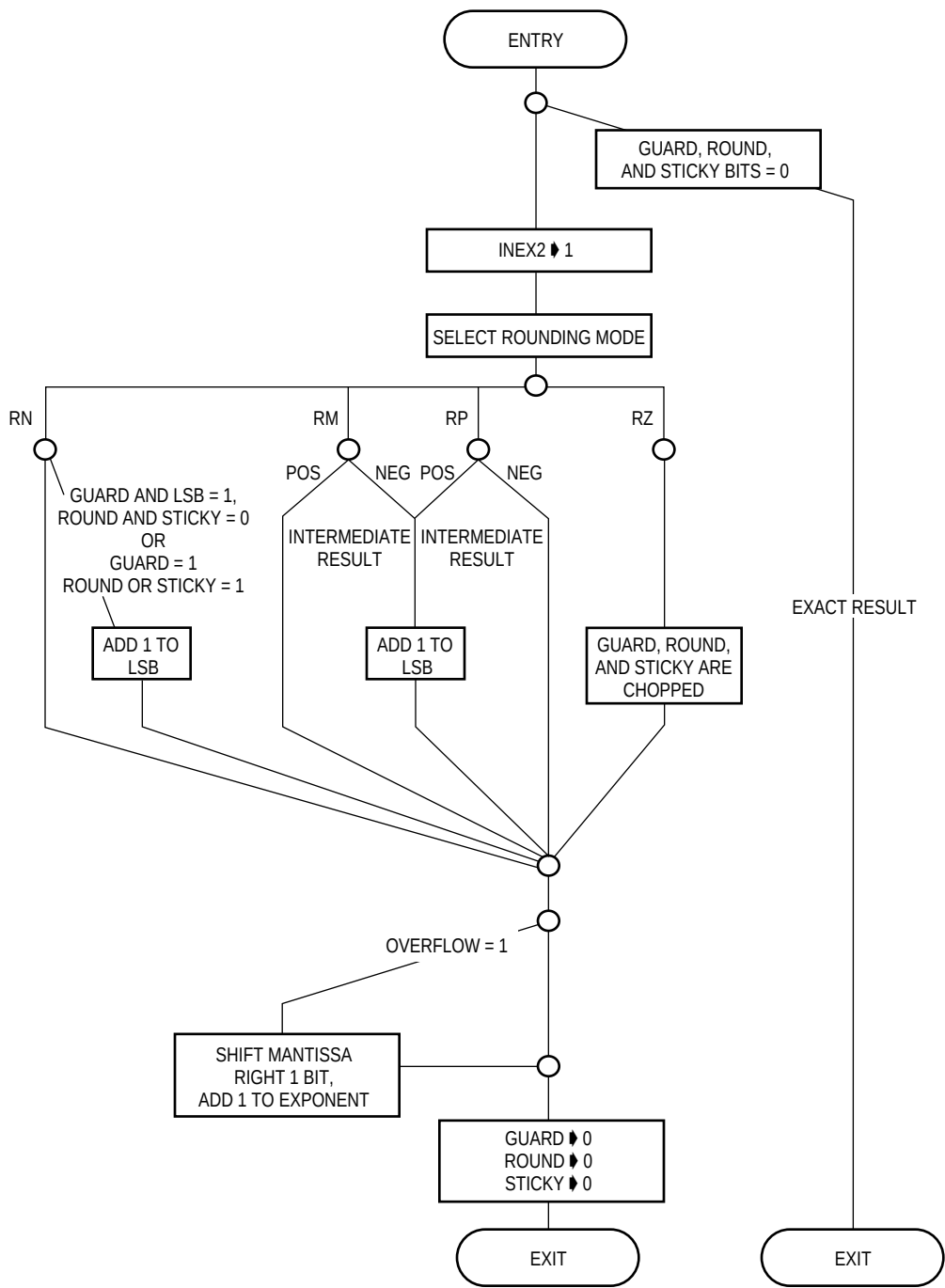


Figure 3-2. Rounding Algorithm Flowchart

The three additional bits beyond the extended-precision format, the difference between the intermediate result's 67-bit mantissa and the storing result's 64-bit mantissa, allow the FPU to perform all calculations as though it were performing calculations using a float engine with infinite bit prec. The result is always correct for the specified destination's data format before performing rounding (unless an overflow or underflow error occurs). The specified rounding operation then produces a number that is as close as possible to the infinitely precise

intermediate value and still representable in the The following tie-case example shows how the 67-bit mantissa allows the FPU to meet the error bound of the IEEE specification:

Result	Integer	63-Bit Fraction	Guard	Round	Sticky
Intermediate	x	xxx...x00	1	0	0
Rounded-to-Nearest	x	xxx...x00	0	0	0

The LSB of the rounded result does not increment though the guard bit is set in the intermediate result. The IEEE 754 standard specifies that tie cases should be handled in this manner. If the destination data format is extended and there is a difference between the infinitely precise intermediate result and the round-to-nearest result, the relative difference is 2^{-64} (the value of the guard bit). This error is equal to half of the least significant bit's value and is the worst case error that can be introduced when using the RN mode. Thus, the term one-half unit in the last place correctly identifies the error bound for this operation. This error specification is the relative error present in the result; the absolute error bound is equal to $2^{\text{exponent}} \times 2^{-64}$. The following example shows the error bound for the other rounding modes:

Result	Integer	63-Bit Fraction	Guard	Round	Sticky
Intermediate	x	xxx...x00	1	1	1
Rounded-to-Nearest	x	xxx...x00	0	0	0

The difference between the infinitely precise result and the rounded result is $2^{-64} + 2^{-65} + 2^{-66}$, which is slightly less than 2^{-63} (the value of the LSB). Thus, the error bound for this operation is not more than one unit in the last place. For all arithmetic operations, the FPU meets these error bounds, providing accurate and repeatable results.

3.6 FLOATING-POINT POSTPROCESSING

Most operations end with a postprocessing step. The FPU provides two steps in postprocessing. First, the condition code bits in the FPSR are set or cleared at the end of each arithmetic operation or move operation to a single floating-point data register. The condition code bits are consistently set based on the result of the operation. Second, the FPU supports 32 conditional tests that allow floating-point conditional instructions to test floating-point conditions in exactly the same way as the integer conditional instructions test the integer condition code. The combination of consistently set condition code bits and the simple programming of conditional instructions gives the processor a very flexible, high-performance method of altering program flow based on floating-point results. While reading the summary for each instruction, it should be assumed that an instruction performs postprocessing unless the summary specifically states that the instruction does not do so. The following paragraphs describe postprocessing in detail.

3.6.1 Underflow, Round, Overflow

During the calculation of an arithmetic result, the FPU arithmetic logic unit (ALU) has more precision and range than the 80-bit extended precision format. However, the final result of these operations is an extended-precision floating-point value. In some cases, an intermediate result becomes either smaller or larger than can be represented in extended precision. Also, the operation can generate a larger exponent or more bits of precision than can be represented in the chosen rounding precision. For these reasons, every arithmetic instruction ends by rounding the result and checking for overflow and underflow.

At the completion of an arithmetic operation, the intermediate result is checked to see if it is too small to be represented as a normalized number in the selected precision. If so, the underflow (UNFL) bit is set in the FPSR EXC byte. It is also denormalized unless denormalization provides a zero value. Denormalizing a number causes a loss of accuracy, but a zero is not returned unless absolutely necessary. If a number is grossly underflowed, the FPU returns a zero or the smallest denormalized number with the correct sign, depending on the rounding mode in effect.

If no underflow occurs, the intermediate result is rounded according to the user-selected rounding precision and rounding mode. After rounding, the inexact bit (INEX2) is set appropriately. Lastly, the magnitude of the result is checked to see if it is too large to be represented in the current rounding precision. If so, the overflow (OVFL) bit is set and a correctly signed infinity or correctly signed largest normalized number is returned, depending on the rounding mode in effect.

3.6.2 Conditional Testing

Unlike the integer arithmetic condition codes, an instruction either always sets the floating-point condition codes in the same way or it does not change them at all. Therefore, the instruction descriptions do not include floating-point condition code settings. The following paragraphs describe how floating-point condition codes are set for all instructions that modify condition codes.

The condition code bits differ slightly from the integer condition codes. Unlike the operation type dependent integer condition codes, examining the result at the end of the operation sets or clears the floating-point condition codes accordingly. The M68000 family integer condition codes bits N and Z have this characteristic, but the V and C bits are set differently for different instructions. The data type of the operation's result determines how the four condition code bits are set. Table 3-22 lists the condition code bit setting for each data type. Loading the FPCC with one of the other combinations and executing a conditional instruction can produce an unexpected branch condition.

Table 3-22. FPCC Encodings

Data Type	N	Z	I	NAN
+ Normalized or Denormalized	0	0	0	0
– Normalized or Denormalized	1	0	0	0
+ 0	0	1	0	0
– 0	1	1	0	0
+ Infinity	0	0	1	0
– Infinity	1	0	1	0
+ NAN	0	0	0	1
– NAN	1	0	0	1

The inclusion of the NAN data type in the IEEE floating-point number system requires each conditional test to include the NAN condition code bit in its Boolean equation. Because a comparison of a NAN with any other data type is unordered (i.e., it is impossible to determine if a NAN is bigger or smaller than an in-range number), the compare instruction sets the NAN condition code bit when an unordered compare is attempted. All arithmetic instructions also set the NAN bit if the result of an operation is a NAN. The conditional instructions interpret the NAN condition code bit equal to one as the unordered condition.

The IEEE 754 standard defines four conditions: equal to (EQ), greater than (GT), less than (LT), and unordered (UN). In addition, the standard only requires the generation of the condition codes as a result of a floating-point compare operation. The FPU can test these conditions at the end of any operation affecting the condition codes. For purposes of the floating-point conditional branch, set byte on condition, decrement and branch on condition, and trap on condition instructions, the processor logically combines the four FPCC condition codes to form 32 conditional tests. There are three main categories of conditional tests: IEEE nonaware tests, IEEE aware tests, and miscellaneous. The set of IEEE nonaware tests is best used:

- when porting a program from a system that does not support the IEEE standard to a conforming system, or
- when generating high-level language code that does not support IEEE floating-point concepts (i.e., the unordered condition).

The 32 conditional tests are separated into two groups; 16 that cause an exception if an unordered condition is present when the conditional test is attempted and 16 that do not cause an exception. An unordered condition occurs when one or both of the operands in a floating-point compare operation. The inclusion of the unordered condition in floating-point branches destroys the familiar trichotomy relationship (greater than, equal, less than) that exists for integers. For example, the opposite of floating-point branch greater than (FBGT) is not floating-point branch less than or equal (FBLE). Rather, the opposite condition is floating-point branch not greater than (FBNGT). If the result of the previous instruction was unordered, FBNGT is true; whereas, both FBGT and FBLE would be false since unordered fails both of these tests (and sets BSUN). Compiler programmers should be particularly careful of the lack of trichotomy in the floating-point branches since it is common for compilers to invert the sense of conditions.

When using the IEEE nonaware tests, the user receives a BSUN exception whenever a branch is attempted and the NAN condition code bit is set, unless the branch is an FBEQ or an FBNE. If the BSUN exception is enabled in the FPCR, the exception causes another exception. Therefore, the IEEE nonaware program is interrupted if an unexpected condition occurs. Compilers and programmers who are knowledgeable of the IEEE 754 standard should use the IEEE aware tests in programs that contain ordered and unordered conditions. Since the ordered or unordered attribute is explicitly included in the conditional test, the BSUN bit is not set in the FPSR EXC byte when the unordered condition occurs. Table 3-23 summarizes the conditional mnemonics, definitions, equations, predicates, and whether the BSUN bit is set in the FPSR EXC byte for the 32 floating-point conditional tests. The equation column lists the combination of FPCC bits for each test in the form of an equation. All condition codes with an overbar indicate cleared bits; all other bits are set.

Table 3-23. Floating-Point Conditional Tests

Mnemonic	Definition	Equation	Predicate	BSUN Bit Set
IEEE Nonaware Tests				
EQ	Equal	Z	000001	No
NE	Not Equal	Z	001110	No
GT	Greater Than	$\overline{NAN} \vee \overline{Z} \vee \overline{N}$	010010	Yes
NGT	Not Greater Than	$NAN \vee Z \vee N$	011101	Yes
GE	Greater Than or Equal	$Z \vee (\overline{NAN} \vee \overline{N})$	010011	Yes
NGE	Not Greater Than or Equal	$NAN \vee (N \wedge \overline{Z})$	011100	Yes
LT	Less Than	$N \wedge (\overline{NAN} \vee \overline{Z})$	010100	Yes
NLT	Not Less Than	$NAN \vee (Z \vee \overline{N})$	011011	Yes
LE	Less Than or Equal	$Z \vee (N \wedge \overline{NAN})$	010101	Yes
NLE	Not Less Than or Equal	$NAN \vee (\overline{N} \vee \overline{Z})$	011010	Yes
GL	Greater or Less Than	$\overline{NAN} \vee \overline{Z}$	010110	Yes
NGL	Not Greater or Less Than	$NAN \vee Z$	011001	Yes
GLE	Greater, Less or Equal	NAN	010111	Yes
NGLE	Not Greater, Less or Equal	NAN	011000	Yes
IEEE Aware Tests				
EQ	Equal	Z	000001	No
NE	Not Equal	Z	001110	No
OGT	Ordered Greater Than	$\overline{NAN} \vee \overline{Z} \vee \overline{N}$	000010	No
ULE	Unordered or Less or Equal	$NAN \vee Z \vee N$	001101	No
OGE	Ordered Greater Than or Equal	$Z \vee (\overline{NAN} \vee \overline{N})$	000011	No
ULT	Unordered or Less Than	$NAN \vee (N \wedge \overline{Z})$	001100	No
OLT	Ordered Less Than	$N \wedge (\overline{NAN} \vee \overline{Z})$	000100	No
UGE	Unordered or Greater or Equal	$NAN \vee Z \vee N$	001011	No
OLE	Ordered Less Than or Equal	$Z \vee (N \wedge \overline{NAN})$	000101	No
UGT	Unordered or Greater Than	$NAN \vee (\overline{N} \vee \overline{Z})$	001010	No
OGL	Ordered Greater or Less Than	$\overline{NAN} \vee \overline{Z}$	000110	No
UEQ	Unordered or Equal	$NAN \vee Z$	001001	No
OR	Ordered	NAN	000111	No
UN	Unordered	NAN	001000	No
Miscellaneous Tests				
F	False	False	000000	No
T	True	True	001111	No
SF	Signaling False	False	010000	Yes
ST	Signaling True	True	011111	Yes
SEQ	Signaling Equal	Z	010001	Yes
SNE	Signaling Not Equal	Z	011110	Yes

3.7 INSTRUCTION DESCRIPTIONS

Section 4, 5, 6, and 7 contain detailed information about each instruction in the M68000 family instruction set. Each section arranges the instruction in alphabetical order by instruction mnemonic and includes descriptions of the instruction's notation and format. Figure 3-3 illustrates the format of the instruction descriptions. Note that the illustration is an amalgamation of the various parts that make up an instruction description. Instruction descriptions for the integer unit differ slightly from those for the floating-point unit; i.e. there are no operation tables included for integer unit instruction descriptions.

The size attribute line specifies the size of the operands of an instruction. When an instruction uses operands of more than one size, the mnemonic of the instruction includes a suffix such as:

- .B—Byte Operands
- .W—Word Operands
- .L—Long-Word Operands
- .S—Single-Precision Real Operands
- .D—Double-Precision Real Operands
- .X—Extended-Precision Real Operands
- .P—Packed BCD Real Operands

The instruction format specifies the bit pattern and fields of the operation and command words, and any other words that are always part of the instruction. The effective address extensions are not explicitly illustrated. The extension words, if any, follow immediately after the illustrated portions of the instructions.

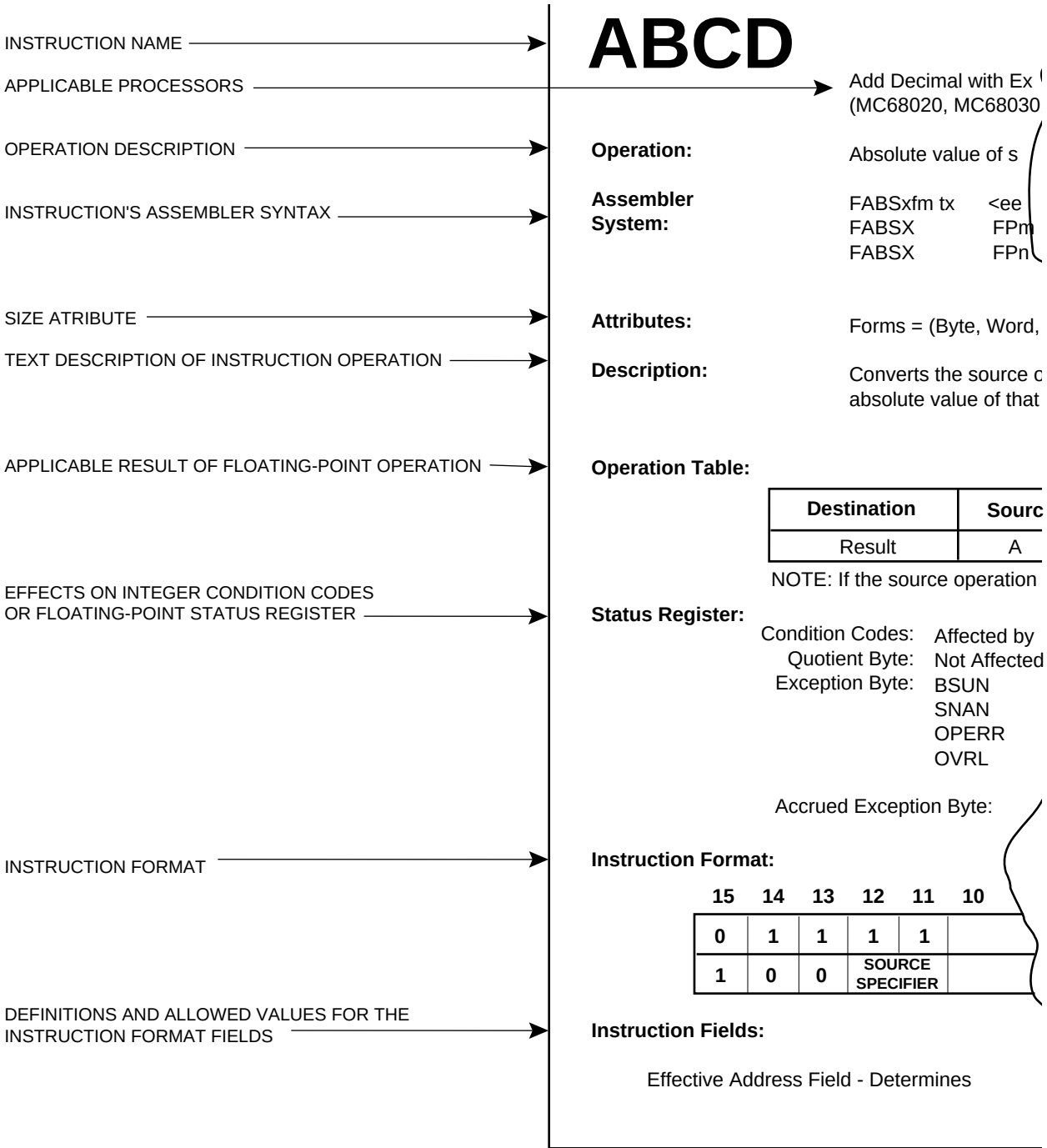


Figure 3-3. Instruction Description Format